



Definición de prueba de Built-in Functional Interfaces

Descripción general

1. Tiempo máximo de realización para la realización: 30 minutos
2. Se realizará el jueves 24/09/2026
3. Es una prueba cuyo resultado no pondera para la obtención de la nota del módulo.
4. La prueba se realizará en papel

5. Para la realización de la prueba se dispondrá de la siguiente tabla para su consulta.

nal Interfaces	# Parameters	Return Type	Single Abstract M
er<T>	0	T	get
er<T>	1 (T)	void	accept
umer<T, U>	2 (T, U)	void	accept
ate<T>	1 (T)	boolean	test
icate<T, U>	2 (T, U)	boolean	test
on<T, R>	1 (T)	R	apply
tion<T, U, R>	2 (T, U)	R	apply
perator<T>	1 (T)	T	apply
Operator<T>	2 (T, T)	T	apply

6. Habrá preguntas de distinta naturaleza, distribuidas de la siguiente forma y puntuación:
- Bloque 1: Preguntas de creación de variables de tipo Build-in funcional interface
 - 4 preguntas 0,75 puntos cada una.
 - Bloque 2: Preguntas sobre las interfaces funcionales: diferencias y/o similitudes.
 - 2 preguntas 1 punto cada una.
 - Bloque 3: Simplificar lambdas
 - 5 preguntas 0,5 puntos cada una.
 - Bloque 4: Escribir las lambdas completas a partir de versiones simplificadas
 - 5 preguntas 0,5 puntos cada una.

Ejemplos de preguntas

▼ Bloque 1

1. Crea una variable llamada `lambda1a` de tipo Built-In Functional Interface compatible con la versión simplificada de la expresión lambda `() → "Hola Mundo"`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable que definirás llamada `lambda1b`.
2. Crea una variable llamada `lambda2a` de tipo Built-In Functional Interface compatible con la versión simplificada de la expresión lambda `(a, b) → a + b`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable que definirás llamada `lambda2b`.
3. Crea una variable llamada `lambda3a` de tipo Built-In Functional Interface compatible con la versión simplificada de la expresión lambda `str → str.length() > 5`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable que definirás llamada `lambda3b`.
4. Crea una variable llamada `lambda4a` de tipo Built-In Functional Interface compatible con la versión simplificada de la expresión lambda `(x, y) → x.compareTo(y)`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable que definirás llamada `lambda4b`.
5. Crea una variable llamada `lambda5a` de tipo Built-In Functional Interface compatible con la versión simplificada de la expresión lambda `lista → lista.isEmpty()`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable que definirás llamada `lambda5b`.
6. Crea una variable llamada `lambda6a` de tipo Built-In Functional Interface compatible con la expresión lambda `nombre → nombre.toUpperCase()`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable llamada `lambda6b`.
7. Crea una variable llamada `lambda7a` de tipo Built-In Functional Interface compatible con la expresión lambda `() → 100`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnalo a una variable llamada `lambda7b`.

8. Crea una variable llamada `lambda8a` de tipo Built-In Functional Interface compatible con la expresión lambda `mensaje -> System.out.println(mensaje)`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnala a una variable llamada `lambda8b`.
9. Crea una variable llamada `lambda9a` de tipo Built-In Functional Interface compatible con la expresión lambda `(a, b) -> a > b`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnala a una variable llamada `lambda9b`.
10. Crea una variable llamada `lambda10a` de tipo Built-In Functional Interface compatible con la expresión lambda `(a, b) -> a * b`, y expresa la misma función lambda en su formato completo (no simplificado) y asígnala a una variable llamada `lambda10b`.

▼ Bloque 2

1. ¿Con que otro Built-In Functional Interface relacionarías `UnaryOperator` y cual es la relación?
2. ¿Qué relación hay entre un `Consumer` y un `BiConsumer`? Pon un ejemplo de `BiConsumer`.
3. ¿Qué relación hay entre un `Predicate` y un `BiPredicate`? Pon un ejemplo de `BiPredicate`.
4. ¿Qué relación existe entre `Function<T,R>` y `BiFunction<T,U,R>`? Explica qué diferencia existe entre ambas respecto al número de parámetros de entrada.
5. ¿Qué relación existe entre `UnaryOperator<T>` y `Function<T,R>`? ¿Qué característica debe cumplir el tipo recibido y el tipo devuelto por un `UnaryOperator`?
6. ¿Qué relación existe entre `BinaryOperator<T>` y `BiFunction<T,U,R>`? Pon un ejemplo de una expresión lambda que pueda asignarse a un `BinaryOperator<Integer>`.
7. ¿Qué diferencias existen entre `Predicate<T>` y `Function<T,R>`? ¿Podría una misma expresión lambda ser compatible con ambas interfaces? Justifica la respuesta.
8. ¿Qué tienen en común `Consumer<T>` y `Function<T,R>` y cuál es su principal diferencia?
9. ¿Qué diferencias existen entre `Supplier<T>` y `Consumer<T>` respecto a los parámetros que reciben y los valores que devuelven?
10. Observa `Predicate<String>` y `Function<String, Boolean>`. ¿Qué similitudes tienen? ¿Qué diferencias existen entre ambas interfaces?

▼ Bloque 3

```
/* 01 */ (int s) -> { return s + 1; } /* 02 */ (int x, int y) -> {
return x + y; } /* 03 */ () -> { System.out.println("Hola Mundo");
} /* 04 */ (String s) -> { return s.isEmpty(); } /* 05 */ (double
d) -> { return d * 2.0; } /* 06 */ (int a, int b) -> {
System.out.println(a - b); } /* 07 */ (String s1, String s2) -> {
return s1.equals(s2); } /* 08 */ () -> {return 42;} /* 09 */
(List<String> list) -> { return list.size(); } /* 10 */
(Map<String, Integer> map) -> { return map.isEmpty(); } /* 11 */
(char c) -> { return Character.isDigit(c); } /* 12 */ (String str)
-> str.toUpperCase() /* 13 */ (boolean b) -> { return !b; } /* 14
*/ (String s, int n) -> { return s.substring(0, n); } /* 15 */ (int
n, double d) -> { return Math.pow(n, d); }
```

▼ Bloque 4

```
/* 01 */ n -> n + 1 /* 02 */ (a, b) -> a + b /* 03 */ () ->
System.out.println("Hola Mundo") /* 04 */ s -> s.isEmpty() /* 05 */
d -> d * 2.0 /* 06 */ (a, b) -> System.out.println(a - b) /* 07 */
(s1, s2) -> s1.equals(s2) /* 08 */ () -> 42 /* 09 */ list ->
list.size() /* 10 */ map -> map.isEmpty() /* 11 */ str ->
str.toUpperCase() /* 12 */ b -> !b /* 13 */ (s, n) ->
s.substring(0, n) /* 14 */ (n, d) -> Math.pow(n, d)
```

Soluciones

▼ Soluciones bloque 1

1. Built-in Functional Interface: `Supplier<String>`

```
Supplier<String> lambda1a = () -> "Hola Mundo"; Supplier<String>  
lambda1b = () -> { return "Hola Mundo"; };
```

2. Built-in Functional Interface: `BinaryOperator<Integer>`

```
BinaryOperator<Integer> lambda2a = (a, b) -> a + b;  
BinaryOperator<Integer> lambda2b = (Integer a, Integer b) -> {  
    return a + b; };
```

3. Built-in Functional Interface: `Predicate<String>`

```
Predicate<String> lambda3a = str -> str.length() > 5;  
Predicate<String> lambda3b = (String str) -> { return str.length()  
> 5; };
```

4. Built-in Functional Interface: `Comparator<String>`

```
Comparator<String> lambda4a = (x, y) -> x.compareTo(y);  
Comparator<String> lambda4b = (String x, String y) -> { return  
x.compareTo(y); };
```

5. Built-in Functional Interface: `Predicate<List<String>>`

```
Predicate<List<String>> lambda5a = lista -> lista.isEmpty();  
Predicate<List<String>> lambda5b = (List<String> lista) -> { return  
lista.isEmpty(); };
```

6. Built-in Functional Interface: `UnaryOperator<String>`

```
UnaryOperator<String> lambda6a = nombre -> nombre.toUpperCase();  
UnaryOperator<String> lambda6b = (String nombre) -> { return  
nombre.toUpperCase(); };
```

7. Built-in Functional Interface: `Supplier<Integer>`

```
Supplier<Integer> lambda7a = () -> 100; Supplier<Integer> lambda7b  
= () -> { return 100; };
```

8. Built-in Functional Interface: `Consumer<String>`

```
Consumer<String> lambda8a = mensaje -> System.out.println(mensaje);  
Consumer<String> lambda8b = (String mensaje) -> {  
System.out.println(mensaje); };
```

9. Built-in Functional Interface: `BiPredicate<Integer, Integer>`

```
BiPredicate<Integer, Integer> lambda9a = (a, b) -> a > b;  
BiPredicate<Integer, Integer> lambda9b = (Integer a, Integer b) ->  
{ return a > b; };
```

10. Built-in Functional Interface: `BinaryOperator<Integer>`

▼ Soluciones bloque 2

```
BinaryOperator<Integer> lambda10a = (a, b) -> a * b;
```

1. Se relaciona con `Function<T,R>`. `BinaryOperator<T>` es una especialización de `Function<T,T>` en la que el tipo de entrada y el de salida son el mismo (equivale a `Function<T,T>`).

2. `BiConsumer<T,U>` es la versión de dos parámetros de `Consumer<T>`: recibe dos valores de entrada en vez de uno y, como `Consumer`, no devuelve nada. Ejemplo: `BiConsumer<String, Integer> imprimir = (nombre, edad) -> System.out.println(nombre + " tiene " + edad + " años");`

3. `BiPredicate<T,U>` es la versión de dos parámetros de `Predicate<T>`: recibe dos valores de entrada en vez de uno y, como `Predicate`, devuelve un `boolean`. Ejemplo: `BiPredicate<String, Integer> tieneLongitud = (texto, longitud) -> texto.length() == longitud;`

4. `BiFunction<T,U,R>` es la versión de dos parámetros de `Function<T,R>`. `Function` recibe un único parámetro de tipo `T` y devuelve un `R`; `BiFunction` recibe dos parámetros, de tipos `T` y `U` (que pueden ser distintos entre sí), y también devuelve un `R`.